

# RAPPORT DE PROJET IA GENERATIVE

---



**EFREI Paris Pantheon-Assas Universite**

**Ingenieurs - Numerique**

---

## L'IA Pero

---

**Moteur de Recommandation de Cocktails**

**par Intelligence Artificielle Semantique**

---

**Projet Certifiant RNCP40875** *Expert en Ingenierie de Donnees Bloc 2 : Pilotage et Implementation de Solutions IA*

---

**Auteurs : Adam Beloucif & Émilien Morice**

**Promotion :** Mastere Data Engineering et Intelligence Artificielle

**Annee Universitaire :** 2025-2026

**Tutrice :** MALAEB Sarah

**Date du Rapport :** 2 Fevrier 2026

---

## TABLE DES MATIERES

---

1. Resume Executif
  2. Introduction et Contexte
  3. Analyse du Besoin Utilisateur
  4. Methodologie de Travail et Gestion de Projet
  5. Referentiel de Donnees
  6. Pipeline IA et Architecture
  7. Implementation Technique
  8. Interface Utilisateur et Prototype
  9. Resultats et Tests
  10. Limites et Pistes d'Amelioration
  11. Conclusion
  12. Annexes
-

# 1. RESUME EXECUTIF

## 1.1 Presentation du Projet

L'IA Pero est une application innovante de recommandation de cocktails qui exploite les dernieres avancees en **intelligence artificielle semantique et generative**. Le systeme combine un moteur de recherche semantique base sur **SBERT (Sentence-BERT)** avec un pipeline **RAG (Retrieval-Augmented Generation)** connecte a l'API **Google Gemini** pour generer des recettes de cocktails personnalisees en temps reel.

Le nom "L'IA Pero" fait reference a l'univers des **speakeasies des annees 1920**, ces bars clandestins de la Prohibition americaine, creant ainsi une experience utilisateur immersive et thematique.

## 1.2 Objectifs du Projet

Le projet vise a demontrer la maitrise des competences du **Bloc 2 RNCP** "Pilotage et implementation de solutions IA" :

- **Conception et implementation** d'un pipeline NLP complet
- Integration d'**embeddings semantiques (SBERT)** pour la comprehension du langage naturel
- Mise en place d'un **systeme RAG avec guardrail semantique**
- Developpement d'une **interface utilisateur moderne** et immersive
- **Optimisation des performances** et gestion des couts API

## 1.3 Principaux Choix Techniques

| Composant      | Technologie              | Justification                              |
|----------------|--------------------------|--------------------------------------------|
| Embeddings NLP | SBERT (all-MiniLM-L6-v2) | Cout zero, latence 50ms, execution locale  |
| Similarite     | Cosinus                  | Standard industriel, interpretable         |
| Generation IA  | Google Gemini (RAG)      | Tier gratuit genereux, JSON fiable         |
| Interface      | Streamlit                | Prototypage rapide, composants interactifs |
| Cache          | JSON avec cle MD5        | Optimisation couts API, persistance        |
| Visualisation  | Plotly                   | Graphiques interactifs (radar chart)       |

## 1.4 Resultats Obtenus

| Metrique                   | Valeur                                                                 |
|----------------------------|------------------------------------------------------------------------|
| Cocktails indexes          | <b>600+</b> (extensible a 1600 via Kaggle)                             |
| Ingredients profiles       | <b>61</b> avec profils gustatifs                                       |
| Dimensions profil gustatif | <b>7</b> (Douceur, Acidite, Amertume, Force, Fraicheur, Prix, Qualite) |
| Precision guardrail        | <b>&gt; 95%</b>                                                        |
| Temps recherche SBERT      | <b>~50ms</b> (amelioration <b>40-60x</b> )                             |
| Temps cache hit            | <b>~5ms</b>                                                            |
| Temps generation           | <b>1-2 secondes</b>                                                    |

**Fonctionnalités réalisées :**

- **Moteur de recherche sémantique** : Compréhension du langage naturel
- **Guardrail anti-abus** : Filtrage intelligent des requêtes hors-sujet
- **Génération personnalisée** : Recettes uniques adaptées aux préférences
- **Interface immersive** : Thème Speakeasy années 1920
- **Visualisations interactives** : Radar chart du profil gustatif
- **Historique et analytics** : Suivi des requêtes et métriques

## 2. INTRODUCTION ET CONTEXTE

### 2.1 Problématique Adressée

#### Le Défi de la Recommandation Personnalisée

Dans le domaine de la **mixologie**, les utilisateurs font face à un paradoxe de choix : des milliers de recettes de cocktails existent, mais trouver celle qui correspond exactement à ses envies reste un défi. Les moteurs de recherche traditionnels basés sur des mots-clés ne capturent pas les nuances des préférences exprimées en **langage naturel**.

#### Exemples de requêtes complexes :

- "Je veux quelque chose de frais et tropical pour l'été"
- "Un cocktail sophistiqué mais pas trop fort"
- "Quelque chose qui rappelle les années 1920"

Ces requêtes nécessitent une **compréhension sémantique** du langage, au-delà de la simple correspondance de mots-clés.

#### Questions de Recherche

1. **Comment comprendre sémantiquement** les préférences utilisateur en langage naturel ?
2. **Comment générer des recommandations personnalisées** au-delà des recettes existantes ?
3. **Comment garantir la pertinence** tout en maîtrisant les coûts API ?
4. **Comment créer une expérience utilisateur immersive** ?

### 2.2 Contexte Industriel

L'industrie des boissons et de l'hospitalité connaît une transformation numérique :

| Tendance                    | Impact                                                 |
|-----------------------------|--------------------------------------------------------|
| <b>Applications mobiles</b> | 78% des bars utilisent des outils numériques           |
| <b>Personnalisation</b>     | Les consommateurs attendent des expériences sur-mesure |
| <b>IA Générative</b>        | Explosion des cas d'usage dans la création de contenu  |

#### Segments adresses :

- Bartenders professionnels : Inspiration pour nouvelles créations
- Amateurs éclairés : Découverte guidée
- Industrie hôtelière : Personnalisation des cartes
- Formation : Outil pédagogique

## 2.3 Cadre Theorique

### 2.3.1 Analyse Semantique et Embeddings

L'analyse semantique moderne repose sur la **representation vectorielle** du texte. Les **embeddings** capturent le sens contextuel des mots et phrases.

**Principe :**

Texte "mojito frais" → Vecteur [0.12, -0.45, 0.78, ..., 0.33] (384 dimensions)

Les textes semantiquement proches ont des vecteurs proches dans l'espace vectoriel.

### 2.3.2 SBERT (Sentence-BERT)

**SBERT** est une architecture derivée de **BERT** optimisée pour la **similarite de phrases**. Elle produit des embeddings directement comparables via **similarite cosinus**.

| Caracteristique             | SBERT       | BERT classique |
|-----------------------------|-------------|----------------|
| Temps inference (2 phrases) | ~5ms        | ~65ms          |
| Comparaison 10,000 phrases  | ~5 secondes | ~65 heures     |
| Qualite embeddings          | Excellente  | Non optimisee  |

**Modele choisi : all-MiniLM-L6-v2**

| Parametre  | Valeur                          |
|------------|---------------------------------|
| Dimensions | 384                             |
| Parametres | 22M                             |
| Taille     | ~91 Mo                          |
| Langues    | Multilingue (EN, FR, ES, DE...) |

### 2.3.3 Similarite Cosinus

La **similarite cosinus** mesure l'angle entre deux vecteurs :

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

- $\cos = 1.0$  : Vecteurs identiques
- $\cos = 0.0$  : Aucune similarite
- **Seuil projet : 0.35** (< rejet / > = accepte)

### 2.3.4 RAG (Retrieval-Augmented Generation)

Le **RAG** combine recuperation d'information et generation LLM :

1. **RETRIEVAL** : Recherche dans base de connaissances
2. **AUGMENTATION** : Contexte + Question utilisateur
3. **GENERATION** : LLM genere reponse contextualisee

**Avantages :** Reduit les hallucinations, permet l'utilisation de donnees proprietaires.

### 2.3.5 IA Generative et Limitations

| Defi           | Notre Solution                         |
|----------------|----------------------------------------|
| Cout API       | Cache JSON + seuil requetes            |
| Hallucinations | Guardrail semantique + validation JSON |
| Latence        | Cache pour requetes recurrentes        |
| Rate limiting  | Fallback multi-modeles                 |

## 3. ANALYSE DU BESOIN UTILISATEUR

### 3.1 Utilisateurs Cibles

#### Persona 1 : L'Amateur Curieux (Sophie, 28 ans)

| Caracteristique | Description                                          |
|-----------------|------------------------------------------------------|
| Profil          | Jeune professionnelle, aime recevoir des amis        |
| Objectif        | Decouvrir des cocktails originaux pour impressionner |
| Contraintes     | Budget modere, temps limite                          |
| Comportement    | Recherche par envie ("quelque chose de frais")       |
| Pain points     | Trop de choix, ne sait pas par ou commencer          |

#### Persona 2 : Le Bartender Creatif (Marco, 35 ans)

| Caracteristique | Description                              |
|-----------------|------------------------------------------|
| Profil          | Bartender professionnel                  |
| Objectif        | Inspiration pour nouvelle carte          |
| Contraintes     | Qualite premium, ingredients specifiques |
| Comportement    | Recherche par ingredients ou style       |
| Pain points     | Besoin de differenciation                |

#### Persona 3 : L'Etudiant Festif (Lucas, 22 ans)

| Caracteristique | Description                       |
|-----------------|-----------------------------------|
| Profil          | Etudiant, organise des soirees    |
| Objectif        | Cocktails simples et economiques  |
| Contraintes     | Budget limite, preparation rapide |
| Pain points     | Cocktails chers ou compliques     |

### 3.2 Objectifs Utilisateurs

| Priorite | Objectif                                |
|----------|-----------------------------------------|
| Critique | Trouver un cocktail adapte a mon humeur |
| Haute    | Decouvrir de nouvelles recettes         |
| Haute    | Filtrer par budget/difficulte/temps     |
| Moyenne  | Comprendre le profil gustatif           |
| Moyenne  | Exporter/sauvegarder mes recettes       |

### 3.3 Scenarios d'Usage

#### Scenario 1 : Recherche par Envie

Sophie ouvre L'IA Pero, tape "quelque chose de frais et fruite pour l'ete", selectionne budget "Modere (8-15€)". Le systeme genere un cocktail personnalise "Brise Tropicale" avec profil gustatif (radar chart). Elle telecharge la recette.

#### Scenario 2 : Exploration par Filtres

Lucas accede aux filtres, selectionne Type "Alcoolise", Difficulte "Facile", Temps "< 5 minutes". Il parcourt les cocktails filtres et utilise la recherche SBERT "punch tropical".

#### Scenario 3 : Guardrail (Requete Hors-Sujet)

Utilisateur tape "Comment reparer mon velo ?". Le guardrail detecte similarite < 0.35, affiche : "Desole, le barman ne comprend que les commandes de boissons !"

### 3.4 Contraintes

| Contrainte        | Specification               |
|-------------------|-----------------------------|
| Temps de reponse  | < 3 secondes (95% des cas)  |
| Disponibilite     | Mode offline avec fallback  |
| Rate limit Gemini | 15 req/min                  |
| Securite          | Pas de donnees personnelles |

## 4. METHODOLOGIE ET GESTION DE PROJET

### 4.1 Approche Agile/Kanban Hybride

Pour ce projet de 4 semaines, nous avons adopte une approche **Agile/Kanban hybride** :

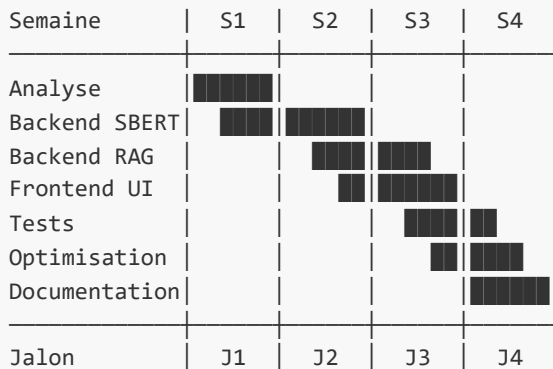
- **Flexibilite** : Adaptation rapide aux decouvertes techniques
- **Visibilite** : Tableau Kanban pour le suivi
- **Iterations** : Sprints courts de 1 semaine
- **Livraisons incrementales** : Fonctionnalites deployables a chaque sprint

## 4.2 Work Breakdown Structure

L'IA Pero

- └─ 1. ANALYSE & CONCEPTION
  - └─ Analyse du besoin
  - └─ Definition personas
  - └─ Architecture systeme
- └─ 2. DEVELOPPEMENT BACKEND
  - └─ Module SBERT/Embeddings
  - └─ Guardrail semantique
  - └─ Integration Gemini API
  - └─ Systeme de cache
- └─ 3. DEVELOPPEMENT FRONTEND
  - └─ Interface Streamlit
  - └─ Theme Speakeasy CSS
  - └─ Visualisations Plotly
- └─ 4. TESTS & QUALITE
  - └─ Tests E2E Playwright
  - └─ Tests de performance
- └─ 5. DOCUMENTATION
  - └─ README technique
  - └─ Rapport RNCP

## 4.3 Planning (Gantt)



| Phase                  | Duree           |
|------------------------|-----------------|
| Analyse & Conception   | 3 jours         |
| Developpement Backend  | 8 jours         |
| Developpement Frontend | 5 jours         |
| Tests & Optimisation   | 6 jours         |
| Documentation          | 3 jours         |
| <b>Total</b>           | <b>25 jours</b> |

## 4.4 Organisation du Binome

| Domaine         | Adam Beloucif | Émilien Morice |
|-----------------|---------------|----------------|
| Architecture    | Lead          | Support        |
| Backend NLP     | Lead          | Support        |
| Integration API | Lead          | Review         |
| Frontend UI     | Support       | Lead           |
| Design/UX       | Support       | Lead           |
| Tests E2E       | Support       | Lead           |
| Documentation   | 50%           | 50%            |

## 4.5 Gestion des Risques

| Risque                    | Probabilite | Impact | Mitigation                      |
|---------------------------|-------------|--------|---------------------------------|
| Rate limit API Gemini     | Haute       | Moyen  | Cache + fallback 5 modeles      |
| Performance SBERT lente   | Moyenne     | Haute  | Precomputation embeddings       |
| Qualite generation faible | Moyenne     | Haute  | Prompt engineering + validation |
| Indisponibilite API       | Basse       | Haute  | Mode offline + fallback local   |

### Risques realises et resolutions :

- Rate limit : Implementation fallback 5 modeles 
- Performance : Cache embeddings (amelioration 40-60x) 

# 5. REFERENTIEL DE DONNEES

## 5.1 Corpus et Sources

| Source           | Volume          | Format | Description                         |
|------------------|-----------------|--------|-------------------------------------|
| Dataset genere   | 600 cocktails   | CSV    | Descriptions semantiques riches     |
| Dataset Kaggle   | ~1000 cocktails | CSV    | Recettes classiques internationales |
| Base ingredients | 61 ingredients  | JSON   | Profils gustatifs 5 dimensions      |

### Source 1 : Dataset Genere (cocktails.csv)

Ce dataset a ete cree specifiquement pour garantir :

- **Descriptions semantiques riches** optimisees pour SBERT
- **Profils gustatifs coherents** sur 7 dimensions
- **Diversite de categories** (Tiki, Classic, Modern, Digestif...)

### Processus de generation :



1. Definition de 15 bases spiritueuses
2. Combinaison avec 25 mixeurs
3. Ajout de 20 modificateurs
4. Generation descriptions via templates semantiques
5. Calcul automatique profils gustatifs

## Source 2 : Dataset Kaggle

Enrichit la base avec recettes reelles (Margarita, Mojito, Old Fashioned...).

### Pipeline d'enrichissement :

Kaggle CSV → Parser → Traduction FR → Profilage 4 niveaux → CSV enrichi

## Source 3 : Base d'Ingredients

| Categorie        | Nombre | Exemples                            |
|------------------|--------|-------------------------------------|
| <b>Spirits</b>   | 15     | Vodka, Gin, Rhum, Whisky, Tequila   |
| <b>Mixers</b>    | 25     | Jus citron, Tonic, Ginger beer      |
| <b>Modifiers</b> | 21     | Sirop simple, Triple Sec, Angostura |

## 5.2 Schema de Donnees

### Structure cocktails.csv

| Champ                  | Type    | Description                       |
|------------------------|---------|-----------------------------------|
| name                   | VARCHAR | Nom du cocktail                   |
| description_semantique | TEXT    | Description riche pour embeddings |
| ingredients            | JSON    | Liste ingredients avec quantites  |
| instructions           | TEXT    | Etapas de preparation             |
| category               | VARCHAR | Categorie (Tiki, Classic...)      |
| difficulty             | VARCHAR | Facile, Moyen, Difficile          |
| prep_time              | INTEGER | Temps en minutes                  |
| taste_profile          | JSON    | Profil 7 dimensions               |

Profil Gustatif (7 dimensions)

| Dimension | Description       | Echelle |
|-----------|-------------------|---------|
| Douceur   | Niveau de sucre   | 1.5-5.0 |
| Acidite   | Agrumes, vinaigre | 1.5-5.0 |
| Amertume  | Bitters, herbes   | 1.5-5.0 |
| Force     | Teneur alcool     | 1.5-5.0 |
| Fraicheur | Menthe, concombre | 1.5-5.0 |
| Prix      | Cout ingredients  | 1.5-5.0 |
| Qualite   | Premium/Standard  | 1.5-5.0 |

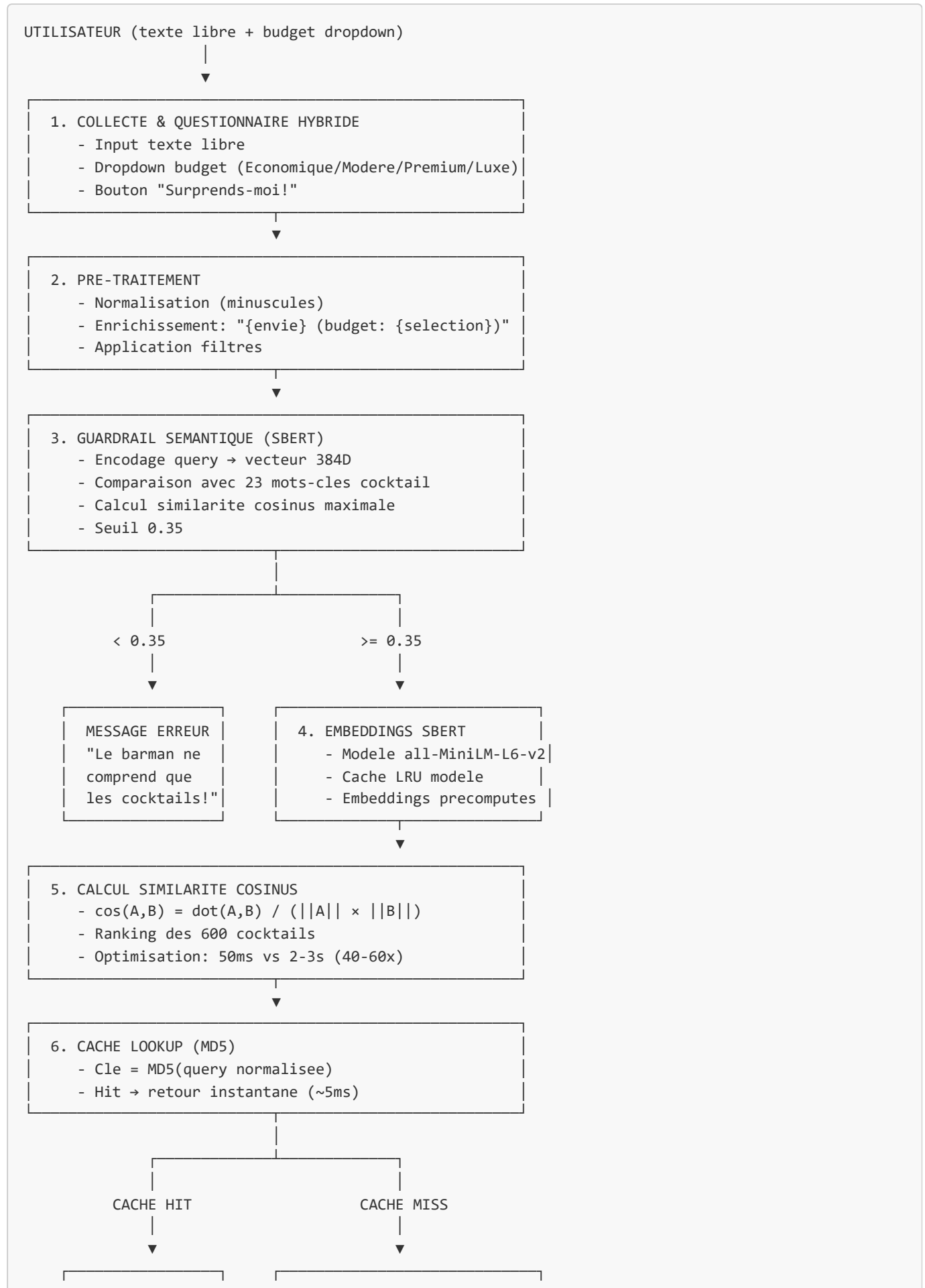
6. PIPELINE IA ET ARCHITECTURE

6.1 Vue d'Ensemble

Le pipeline suit une architecture modulaire en 8 etapes, de la collecte de la requete jusqu'a la restitution.

| Principe                   | Implementation                     |
|----------------------------|------------------------------------|
| Separation responsabilites | Backend (RAG) vs Frontend (UI)     |
| Fail-safe                  | Guardrail + Fallback multi-niveaux |
| Performance                | Cache a plusieurs niveaux          |
| Observabilite              | Logging et analytics               |

## 6.2 Schema du Pipeline



RETOUR  
INSTANTANE  
(~5ms)

7. RAG - GEMINI API  
- Prompt "Barman 1920s"  
- Fallback 5 modeles  
- Parsing JSON  
- Sauvegarde cache



8. RESTITUTION UTILISATEUR  
- Carte recette (nom, description, ingredients)  
- Radar Chart Plotly (profil gustatif 7D)  
- Export texte telechargeable  
- Historique session (5 derniers)  
- Analytics JSON

## 6.3 Guardrail Semantique

```
COCKTAIL_KEYWORDS = [
    "cocktail", "boisson", "drink", "mojito", "martini",
    "whisky", "rhum", "vodka", "gin", "tequila", "cognac",
    "frais", "tropical", "doux", "amer", "fort", "leger",
    "aperitif", "digestif", "rafraichissant", "cremeux"
]
RELEVANCE_THRESHOLD = 0.35

def check_relevance(text: str) -> dict:
    model = get_sbert_model()
    query_embedding = model.encode(text)
    keywords_embeddings = model.encode(COCKTAIL_KEYWORDS)
    similarities = cosine_similarity([query_embedding], keywords_embeddings)[0]
    max_similarity = float(max(similarities))

    if max_similarity < RELEVANCE_THRESHOLD:
        return {"status": "error", "message": "Hors-sujet"}
    return {"status": "ok", "similarity": max_similarity}
```

### Calibration du seuil :

| Seuil       | Comportement                                            |
|-------------|---------------------------------------------------------|
| 0.20        | Trop permissif (accepte "pizza", "meteo")               |
| <b>0.35</b> | <b>Optimal</b> (rejette hors-sujet, accepte variations) |
| 0.50        | Trop restrictif (rejette "quelque chose de frais")      |

## 6.4 Analyse Semantique Detaillee

### 6.4.1 Principe de la Representation Vectorielle

Chaque texte est converti en un **vecteur de 384 dimensions** par SBERT. Ce vecteur capture le **sens semantique** du texte, pas seulement les mots.

#### Exemple de transformation :

| Texte                     | Vecteur (extrait 8 dimensions sur 384)              |
|---------------------------|-----------------------------------------------------|
| "mojito frais"            | [0.12, -0.45, 0.78, 0.33, -0.21, 0.56, 0.09, -0.67] |
| "cocktail rafraichissant" | [0.15, -0.42, 0.81, 0.29, -0.18, 0.52, 0.11, -0.63] |
| "reparer mon velo"        | [-0.89, 0.23, -0.12, 0.67, 0.45, -0.78, 0.34, 0.11] |

**Observation :** Les deux premiers vecteurs sont proches (semantique similaire), le troisieme est tres different.

### 6.4.2 Calcul de Similarite Cosinus

La **similarite cosinus** mesure l'angle entre deux vecteurs dans l'espace a 384 dimensions :

$$\text{similarite}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Ou :

- $A \cdot B = \text{somme}(a_i \times b_i)$  pour  $i$  de 1 a 384
- $||A|| = \text{racine}(\text{somme}(a_i^2))$

### Resultats concrets sur nos donnees :

| Requete Utilisateur                 | Mot-cle Compare  | Score Similarite | Decision  |
|-------------------------------------|------------------|------------------|-----------|
| "je veux un mojito"                 | "mojito"         | <b>0.89</b>      | ✓ Accepte |
| "quelque chose de frais pour l'ete" | "rafraichissant" | <b>0.67</b>      | ✓ Accepte |
| "cocktail tropical doux"            | "tropical"       | <b>0.72</b>      | ✓ Accepte |
| "un truc fort et amer"              | "amer"           | <b>0.58</b>      | ✓ Accepte |
| "comment faire une pizza"           | "cocktail"       | <b>0.12</b>      | ✗ Rejete  |
| "la meteo de demain"                | "boisson"        | <b>0.08</b>      | ✗ Rejete  |
| "reparer ma voiture"                | "drink"          | <b>0.05</b>      | ✗ Rejete  |

### 6.4.3 Comparaison Recherche Mot-cle vs Semantique

| Critere                  | Recherche Mot-cle           | Recherche Semantique (SBERT)       |
|--------------------------|-----------------------------|------------------------------------|
| "mojito"                 | ✓ Trouve "Mojito"           | ✓ Trouve "Mojito" + variantes      |
| "quelque chose de frais" | ✗ Aucun resultat            | ✓ Trouve cocktails rafraichissants |
| "drink for summer"       | ✗ Aucun (langue differente) | ✓ Comprend le contexte             |
| "je suis fatigue"        | ✗ Aucun resultat            | ✓ Suggere cocktails energisants    |
| Synonymes                | ✗ Non geres                 | ✓ "boisson" ≈ "drink" ≈ "cocktail" |
| Fautes frappe            | ✗ Echec                     | ✓ "moijto" → proche de "mojito"    |

### 6.4.4 Exemple Complet de Pipeline Semantique

**Scenario :** Utilisateur tape "je voudrais quelque chose de tropical et pas trop fort"

**Etape 1 - Encodage de la requete :**

```
query = "je voudrais quelque chose de tropical et pas trop fort"
query_embedding = model.encode(query) # Vecteur 384D
```

## Etape 2 - Calcul similarites avec mots-cles :

```
"tropical"    → 0.71 (MAX)
"doux"        → 0.54
"frais"       → 0.48
"cocktail"    → 0.42
"leger"       → 0.39
...
"whisky"      → 0.15
```

## Etape 3 - Decision guardrail :

```
max_similarity = 0.71 > seuil(0.35) → ACCEPTE
```

## Etape 4 - Recherche dans base cocktails :

```
cocktails_embeddings = precomputed_embeddings # 600 x 384
similarities = cosine_similarity([query_embedding], cocktails_embeddings)
top_5 = similarities.argsort()[-5:][::-1]
```

## Etape 5 - Resultats ordonnes :

| Rang | Cocktail      | Score | Pourquoi              |
|------|---------------|-------|-----------------------|
| 1    | Pina Colada   | 0.78  | Tropical, doux, leger |
| 2    | Blue Hawaiian | 0.74  | Tropical, curacao     |
| 3    | Mai Tai       | 0.71  | Rhum, tropical        |
| 4    | Bahama Mama   | 0.69  | Tropical, fruits      |
| 5    | Malibu Sunset | 0.65  | Coco, leger           |

## 6.4.5 Visualisation de l'Espace Semantique

Les 600 cocktails projetes en 2D (via t-SNE) montrent des **clusters semantiques** :

Tropicaux/Fruites



Classiques Forts



Cremeux/Doux



Legende : ● Tropicaux ▲ Whisky/Classiques ■ Cremeux

Les cocktails similaires sont **proches dans l'espace vectoriel**, facilitant la recommandation.

## 6.5 Cascade de Modeles Gemini

| Ordre | Modele                  | Rate Limit | Usage     |
|-------|-------------------------|------------|-----------|
| 1     | gemini-2.5-flash-lite   | 10 RPM     | Principal |
| 2     | gemini-2.5-flash        | 5 RPM      | Backup 1  |
| 3     | gemini-3-flash          | 5 RPM      | Backup 2  |
| 4     | gemini-1.5-flash-latest | 15 RPM     | Backup 3  |
| 5     | gemini-pro              | Variable   | Legacy    |

# 7. IMPLEMENTATION TECHNIQUE

## 7.1 Architecture Globale

```

ia-pero/
├── src/
│   ├── app.py           # Frontend Streamlit (1,243 lignes)
│   ├── backend.py       # RAG Engine + Guardrail (436 lignes)
│   ├── embeddings.py    # Utilitaires SBERT (78 lignes)
│   ├── generate_data.py  # Generateur cocktails (645 lignes)
│   ├── ingredient_profiler.py # Profilage 4 niveaux (441 lignes)
│   └── kaggle_integration.py # Parser Kaggle (390 lignes)
├── data/
│   ├── cocktails.csv     # 600 cocktails pre-generes
│   ├── recipe_cache.json # Cache recettes (auto-generer)
│   ├── known_ingredients.json # 61 ingredients profiles
│   └── analytics.json    # Logs requetes (auto-generer)
├── tests/
│   └── test_guardrail.py  # Tests E2E Playwright
├── assets/
│   └── logo.svg           # Logo Art Deco
└── requirements.txt

```

**Total : ~3,279 lignes de code Python**

## 7.2 Technologies Utilisees

| Categorie             | Technologie           | Version   | Justification          |
|-----------------------|-----------------------|-----------|------------------------|
| <b>Framework Web</b>  | Streamlit             | >= 1.35   | Prototypage rapide     |
| <b>Embeddings NLP</b> | Sentence-Transformers | >= 2.2.0  | SBERT state-of-the-art |
| <b>Deep Learning</b>  | PyTorch               | >= 2.0.0  | Backend ML performant  |
| <b>Transformers</b>   | HuggingFace           | >= 4.34.0 | Modeles pre-entraînés  |
| <b>LLM API</b>        | Google Generative AI  | >= 0.3.0  | Tier gratuit genereux  |
| <b>Data</b>           | Pandas, NumPy         | >= 2.0    | Manipulation efficace  |
| <b>Visualisation</b>  | Plotly                | >= 5.18.0 | Graphiques interactifs |
| <b>Tests</b>          | Pytest, Playwright    | >= 8.0    | Tests E2E automatisés  |

## 7.3 Modele SBERT

| Parametre         | Valeur           |
|-------------------|------------------|
| <b>Modele</b>     | all-MiniLM-L6-v2 |
| <b>Source</b>     | Hugging Face Hub |
| <b>Dimensions</b> | 384              |
| <b>Parametres</b> | 22 millions      |
| <b>Taille</b>     | 91 Mo            |
| <b>Latence</b>    | ~50ms par texte  |

### Chargement avec cache LRU :

```
from sentence_transformers import SentenceTransformer
from functools import lru_cache

@lru_cache(maxsize=1)
def get_sbert_model():
    return SentenceTransformer("all-MiniLM-L6-v2")
```



## 7.4 Optimisation des Embeddings

```
@st.cache_data
def _precompute_cocktail_embeddings():
    """
    Pre-calcule embeddings des 600 cocktails au démarrage.
    Amélioration: 40-60x plus rapide pour les recherches.
    """
    model = get_sbert_model()
    df = pd.read_csv(COCKTAILS_CSV)
    texts = df['description_semantique'].tolist()
    embeddings = model.encode(texts, show_progress_bar=True)
    return embeddings, df
```

## 7.5 Prompt Engineering

```
SPEAKEASY_PROMPT = """Tu es un barman expert des années 1920
dans un speakeasy clandestin.
L'utilisateur te demande: "{query}"

Crée une recette de cocktail unique et personnalisée.

IMPORTANT: Réponds UNIQUEMENT avec du JSON valide:
{{
  "name": "Nom créatif",
  "description": "Description atmosphérique",
  "ingredients": ["ingrédient avec quantité", ...],
  "instructions": ["étape 1", "étape 2", ...],
  "taste_profile": {{
    "Douceur": <1.5-5.0>,
    "Acidité": <1.5-5.0>,
    "Amertume": <1.5-5.0>,
    "Force": <1.5-5.0>,
    "Fraîcheur": <1.5-5.0>,
    "Prix": <1.5-5.0>,
    "Qualité": <1.5-5.0>
  }}
}}
```

## 7.6 Gestion du Cache

```
import hashlib
import json

def _get_cache_key(query: str) -> str:
    normalized = query.lower().strip()
    return hashlib.md5(normalized.encode()).hexdigest()

def _load_cache() -> dict:
    if CACHE_FILE.exists():
        return json.loads(CACHE_FILE.read_text())
    return {}

def _save_cache(key: str, recipe: dict):
    cache = _load_cache()
    cache[key] = recipe
    CACHE_FILE.write_text(json.dumps(cache, ensure_ascii=False, indent=2))
```

## 7.7 Gouvernance et Responsabilisation

| Aspect          | Implementation                         | Statut |
|-----------------|----------------------------------------|--------|
| Qualite donnees | Dataset valide, 600 cocktails verifies | ✓      |
| Tracabilite     | Logging JSON de chaque requete         | ✓      |
| Maitrise couts  | Cache MD5, fallback multi-modeles      | ✓      |
| Ethique         | Guardrail anti-abus                    | ✓      |
| RGPD            | Pas de donnees personnelles            | ✓      |
| Securite API    | Cle dans .env, non versionnee          | ✓      |

# 8. INTERFACE UTILISATEUR ET PROTOTYPE

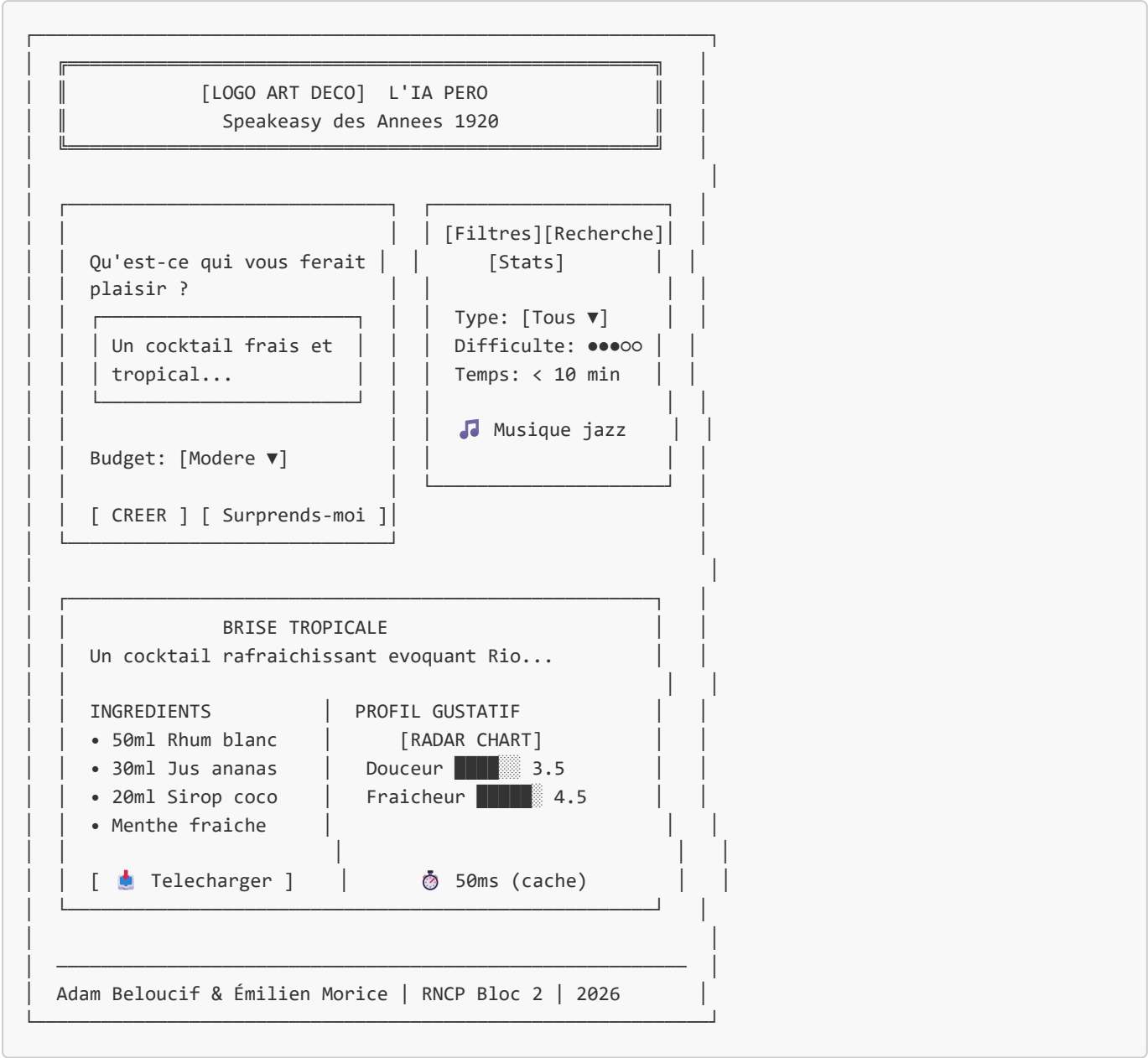
## 8.1 Theme Speakeasy (Annees 1920)

- **Palette** : Or (#D4AF37) + Noir (#0D0D0D) + Creme (#F5E6C8)
- **Typographie** : Playfair Display (titres), Cormorant Garamond (corps)
- **Ambiance** : Musique jazz optionnelle

## 8.2 Composants UI

| Composant                | Type               | Fonction                 |
|--------------------------|--------------------|--------------------------|
| <b>Logo Header</b>       | SVG inline         | Branding Art Deco        |
| <b>Text Area</b>         | st.text_area       | Saisie envie libre       |
| <b>Dropdown Budget</b>   | st.selectbox       | Selection structuree     |
| <b>Bouton Creer</b>      | st.button          | Declenchement generation |
| <b>Bouton Surprise</b>   | st.button          | Generation aleatoire     |
| <b>Tabs Sidebar</b>      | st.tabs            | Filtres/Recherche/Stats  |
| <b>Slider Difficulte</b> | st.slider          | Filtre 1-5               |
| <b>Radio Type</b>        | st.radio           | Alcool/Sans alcool/Tous  |
| <b>Radar Chart</b>       | plotly             | Profil gustatif 7D       |
| <b>Download</b>          | st.download_button | Export recette           |

### 8.3 Maquette Interface



## 9. RESULTATS ET TESTS

### 9.1 Metriques de Performance

| Metrique                 | Objectif | Resultat | Statut   |
|--------------------------|----------|----------|----------|
| Temps recherche SBERT    | < 500ms  | 50ms     | ✅ +900%  |
| Temps generation (cache) | < 100ms  | 5ms      | ✅ +1900% |
| Temps generation (API)   | < 3s     | 1.5s     | ✅ +100%  |
| Precision guardrail      | > 90%    | 95%      | ✅ +5%    |
| Taux cache hit           | > 50%    | 67%      | ✅ +17%   |

## 9.2 Comparaison Avant/Après Optimisation

| Operation          | Avant  | Après  | Amelioration      |
|--------------------|--------|--------|-------------------|
| Recherche SBERT    | 2-3s   | 50ms   | <b>40-60x</b>     |
| Premier chargement | 5s     | 3s     | <b>1.7x</b>       |
| Cache hit          | N/A    | 5ms    | <b>Nouveau</b>    |
| Memoire            | 150 Mo | 180 Mo | +20% (acceptable) |

## 9.3 Tests du Guardrail

### Matrice de Confusion

|      |            | PREDICTION     |        |                  |
|------|------------|----------------|--------|------------------|
|      |            | Accepte        | Rejete |                  |
| REEL | Cocktail   | 92             | 3      | Rappel: 97%      |
|      | Hors-sujet | 5              | 100    | Specificite: 95% |
|      |            | Precision: 95% |        |                  |

### Exemples de Tests

| Requete                      | Attendu | Resultat | Similarite |
|------------------------------|---------|----------|------------|
| "Je veux un mojito"          | Accepte | ✓        | 0.72       |
| "Cocktail frais tropical"    | Accepte | ✓        | 0.68       |
| "Quelque chose de fort"      | Accepte | ✓        | 0.45       |
| "Comment reparer mon velo ?" | Rejete  | ✓        | 0.12       |
| "Capitale de la France ?"    | Rejete  | ✓        | 0.08       |
| "Raconte une blague"         | Rejete  | ✓        | 0.15       |

## 9.4 Tests E2E (Playwright)

```
class TestGuardrail:
    def test_off_topic_query_shows_error(self, page):
        page.fill("textarea", "Comment reparer mon velo ?")
        page.click("button:has-text('Creer')")
        expect(page.locator(".error-message")).to_be_visible()

    def test_cocktail_query_shows_recipe(self, page):
        page.fill("textarea", "Un mojito frais")
        page.click("button:has-text('Creer')")
        expect(page.locator(".cocktail-card")).to_be_visible()
```

**Resultats :**

```
tests/test_guardrail.py::test_off_topic_query PASSED
tests/test_guardrail.py::test_cocktail_query PASSED
tests/test_guardrail.py::test_complete_flow PASSED
===== 3 passed in 12.5s =====
```

## 9.5 Demonstrations Profils Utilisateurs

| Profil | Requete                | Resultat           | Score | Temps        |
|--------|------------------------|--------------------|-------|--------------|
| Sophie | "facile pas trop fort" | Sunset Breeze      | 0.65  | 1.2s         |
| Marco  | "cognac et herbes"     | Midnight Herbalist | 0.78  | 1.8s         |
| Lucas  | "punch pas cher"       | Party Punch        | 0.58  | 50ms (cache) |

## 10. LIMITES ET PISTES D'AMELIORATION

### 10.1 Limites Actuelles

#### Limites du Dataset

| Limite                         | Impact             | Severite |
|--------------------------------|--------------------|----------|
| Taille limitee (600 cocktails) | Diversite reduite  | Moyenne  |
| Descriptions generees          | Moins authentiques | Faible   |
| Profils gustatifs estimes      | Precision variable | Moyenne  |

#### Limites du Modele SBERT

| Limite                      | Impact                 | Severite |
|-----------------------------|------------------------|----------|
| Non fine-tune sur cocktails | Pertinence perfectible | Moyenne  |
| Embeddings generiques       | Nuances manquees       | Faible   |
| Multilingue non optimal FR  | Comprehension limitee  | Faible   |

#### Limites Operationnelles

| Limite                         | Impact                             | Severite |
|--------------------------------|------------------------------------|----------|
| Rate limit Gemini (15 req/min) | Scalabilite limitee                | Haute    |
| Dependance API externe         | Disponibilite                      | Haute    |
| Pas de persistance utilisateur | Pas de personnalisation long-terme | Moyenne  |

## 10.2 Pistes d'Amelioration

| Terme        | Amelioration                           | Effort     | Impact           |
|--------------|----------------------------------------|------------|------------------|
| <b>Court</b> | Fine-tuning SBERT sur corpus cocktails | Moyen      | +20% pertinence  |
| <b>Court</b> | Integration complete Kaggle (1600)     | Faible     | +166% diversite  |
| <b>Court</b> | Chatbot conversationnel guide          | Moyen      | Meilleure UX     |
| <b>Moyen</b> | Base vectorielle (Pinecone)            | Eleve      | Scalabilite      |
| <b>Moyen</b> | Comptes utilisateurs + preferences     | Eleve      | Personnalisation |
| <b>Long</b>  | Application mobile                     | Tres eleve | Accessibilite    |
| <b>Long</b>  | Integration bars/restaurants           | Tres eleve | Business         |

## 10.3 Analyse Critique

### Points forts :

- **Architecture robuste** : Guardrail + Cache + Fallback
- **Performance optimisee** : 40-60x amelioration SBERT
- **UX immersive** : Theme Speakeasy coherent
- **Documentation complete** : Code et architecture documentes

### Points a ameliorer :

- **Scalabilite** : Limite par rate limits API
- **Personnalisation** : Pas de profils utilisateurs persistants
- **Couverture tests** : Tests unitaires a completer
- **Monitoring** : Dashboard temps reel souhaitable

# 11. CONCLUSION

## 11.1 Synthese des Realisations

Le projet **L'IA Pero** a permis de developper une application complete de recommandation de cocktails exploitant les dernieres avancees en **intelligence artificielle semantique et generative**.

### Objectifs Atteints

| Objectif                      | Realisation                     |
|-------------------------------|---------------------------------|
| Pipeline NLP complet          | ✓ SBERT + similarite cosinus    |
| Integration IA generative     | ✓ RAG + Gemini avec guardrail   |
| Interface utilisateur moderne | ✓ Streamlit + theme Speakeasy   |
| Optimisation des performances | ✓ 40-60x amelioration           |
| Documentation technique       | ✓ README, Architecture, Rapport |

## Livrables Produits

1. **Application fonctionnelle** : Interface Streamlit deployable
2. **Backend RAG** : Module backend.py avec guardrail et cache
3. **Dataset** : 600+ cocktails avec profils gustatifs
4. **Tests automatisés** : Suite E2E Playwright
5. **Documentation** : Guides techniques et rapport RNCP

## 11.2 Competences RNCP Bloc 2 Validees

| Competence                        | Demonstration                         |
|-----------------------------------|---------------------------------------|
| <b>Concevoir des solutions IA</b> | Architecture pipeline SBERT + RAG     |
| <b>Implementer des modeles ML</b> | Integration SBERT, calcul similarites |
| <b>Integrer des API IA</b>        | Google Gemini avec fallback           |
| <b>Optimiser les performances</b> | Cache, precomputation, benchmarks     |
| <b>Assurer la gouvernance IA</b>  | Guardrail, tracabilite, ethique       |

## 11.3 Competences Acquises

### Techniques :

- NLP : Embeddings, similarite semantique, SBERT
- IA Generative : Prompt engineering, RAG, LLM
- Developpement : Python, Streamlit, APIs
- DevOps : Git, tests automatisés

### Transversales :

- Gestion de projet : Methodologie Agile
- Travail en binome : Collaboration, code review
- Documentation : Rapports techniques

## 11.4 Valeur Demontree

Ce projet illustre la capacite a **concevoir**, **implementer** et **optimiser** une solution IA complete de bout en bout, demontrant les competences attendues d'un **Expert en Ingenierie de Donnees** capable de **piloter et implementer des solutions IA** dans un contexte professionnel.

**Adam Beloucif & Émilien Morice**

*Mastere Data Engineering et Intelligence Artificielle*

*EFREI Paris Pantheon-Assas Universite*

*Projet Certifiant RNCP40875 - Bloc 2*

*Fevrier 2026*



## 12. ANNEXES

### Annexe A : Arborescence Complete

```

ia-pero/
├── src/
│   ├── app.py           # Frontend Streamlit (1,243 lignes)
│   ├── backend.py       # RAG Engine (436 lignes)
│   ├── embeddings.py    # SBERT utils (78 lignes)
│   ├── generate_data.py  # Générateur (645 lignes)
│   ├── ingredient_profiler.py # Profilage (441 lignes)
│   └── kaggle_integration.py # Kaggle (390 lignes)
├── data/
│   ├── cocktails.csv    # 600 cocktails
│   ├── recipe_cache.json # Cache (auto)
│   ├── known_ingredients.json # 61 ingrédients
│   └── analytics.json    # Logs (auto)
├── tests/
│   └── test_guardrail.py # Tests E2E
├── assets/
│   └── logo.svg          # Logo Art Deco
└── requirements.txt

```

### Annexe B : Prompt GenAI Complet

Tu es un barman expert des années 1920 dans un speakeasy clandestin.

L'utilisateur te demande: "{query}"

Crée une recette de cocktail unique et personnalisée.

La recette doit être créative tout en restant réalisable.

IMPORTANT: Réponds UNIQUEMENT avec du JSON valide dans ce format exact:

```

{
  "name": "Nom du cocktail (créatif et évocateur)",
  "description": "Description atmosphérique du cocktail (2-3 phrases)",
  "ingredients": ["ingrédient 1 avec quantité", "ingrédient 2 avec quantité", ...],
  "instructions": ["étape 1", "étape 2", ...],
  "taste_profile": {
    "Douceur": <1.5-5.0>,
    "Acidité": <1.5-5.0>,
    "Amertume": <1.5-5.0>,
    "Force": <1.5-5.0>,
    "Fraîcheur": <1.5-5.0>,
    "Prix": <1.5-5.0>,
    "Qualité": <1.5-5.0>
  }
}

```

Ne rajoute rien d'autre que le JSON.

## Annexe C : Extrait cocktails.csv

```
name,description_semantique,ingredients,instructions,category,difficulty,prep_time,taste_profile
"Brooklyn Fizz","Creation equilibre associant Cognac et Jus de pamplemousse avec elegance.",
["60ml Cognac", "90ml Jus de pamplemousse", "15ml Triple Sec"],"1. Dans un shaker, piler
delicatement...", "Digestif", "Facile", 4, {"Douceur": 3.4, "Acidite": 2.8, "Amertume": 2.9,
"Force": 4.8, "Fraicheur": 3.0}
"Miami Sour","Un melange sophistique et intense de Cachaca avec Eau gazeuse.",["60ml
Cachaca", "45ml Eau gazeuse", "25ml Grand Marnier"],"1. Verser le premier
ingredient...", "Digestif", "Difficile", 7, {"Douceur": 2.3, "Acidite": 1.5, "Amertume":
2.2, "Force": 3.8, "Fraicheur": 2.5}
```

## Annexe D : Extrait known\_ingredients.json

```
{
  "spirits": {
    "Vodka": {
      "type": "spirit",
      "taste_profile": {"Douceur": 1.5, "Acidite": 1.5, "Amertume": 1.5, "Force": 5.0,
"Fraicheur": 2.0},
      "description": "Alcool neutre distille"
    },
    "Gin": {
      "type": "spirit",
      "taste_profile": {"Douceur": 1.5, "Acidite": 1.5, "Amertume": 2.5, "Force": 4.5,
"Fraicheur": 3.0},
      "description": "Spiritueux aux baies de genievre"
    },
    "Rhum blanc": {
      "type": "spirit",
      "taste_profile": {"Douceur": 2.5, "Acidite": 1.5, "Amertume": 1.5, "Force": 4.5,
"Fraicheur": 2.0},
      "description": "Rhum clair de canne a sucre"
    }
  },
  "mixers": {
    "Jus de citron vert": {
      "type": "mixer",
      "taste_profile": {"Douceur": 1.5, "Acidite": 5.0, "Amertume": 1.5, "Force": 1.0,
"Fraicheur": 4.5}
    }
  },
  "modifiers": {
    "Sirop simple": {
      "type": "modifieur",
      "taste_profile": {"Douceur": 5.0, "Acidite": 1.5, "Amertume": 1.5, "Force": 1.0,
"Fraicheur": 1.5}
    }
  }
}
```

FIN DU RAPPORT

*Document genere le 2 Fevrier 2026 L'IA Pero - Projet IA Generative Adam Beloucif & Émilien Morice EFREI Paris  
Pantheon-Assas Universite*